Exp No: 12	*End-End Communication at Transport Layer - Echo Client Server and Chat*

AIM:

To implement (a) an echo client-server using TCP and UDP sockets and (b) a chat program using socket programming, for end-to-end communication at the transport layer.

ALGORITHM / PROCEDURE:

(a1) TCP echo server and client

1. ***Server***: create a TCP socket with socket(AF_INET, SOCK_STREAM), bind() it to an IP address and port (5000), call listen() and wait in accept() for the client.
2. ***Client***: create a TCP socket and call connect() with the IP address and port of the server.
3. The client sends a message using send(). The server reads it with recv(), displays it and sends the same data back using sendall().
4. The client receives the echoed message and displays it. This is repeated for each message.
5. Close the sockets using close().

(a2) UDP echo server and client

1. ***Server***: create a UDP socket with SOCK_DGRAM and bind() it to port 6000. No `listen()` / `accept()` is needed because UDP is connectionless.
2. ***Client***: create a UDP socket and send the message to the server address with sendto().
3. The server receives the datagram and the client address with recvfrom() and sends it back using `sendto()`.
4. The client receives the echo with recvfrom() and displays it. Close the sockets.

(b) Chat program

1. The chat server creates a TCP socket on port 7000 and waits for a client. The client connects to it.
2. The client types a message and sends it. The server displays it, then the server user types the reply and sends it back.
3. Messages are exchanged alternately until either side sends the word exit. Then both sockets are closed.

Point	***TCP***	***UDP***
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection	Connection oriented (3-way handshake)	Connectionless
Reliability	Reliable, ordered delivery	No guarantee
Server calls	bind, listen, accept, recv, send	bind, recvfrom, sendto
Used in	HTTP, FTP, SMTP	DNS, DHCP, streaming

DIAGRAM:

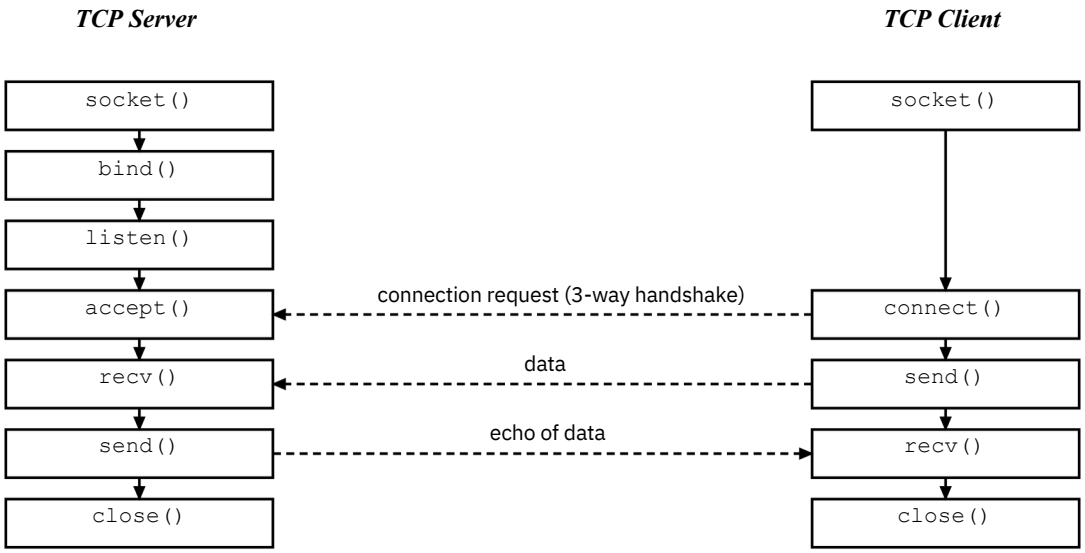


Fig 1: Socket calls in TCP client-server communication

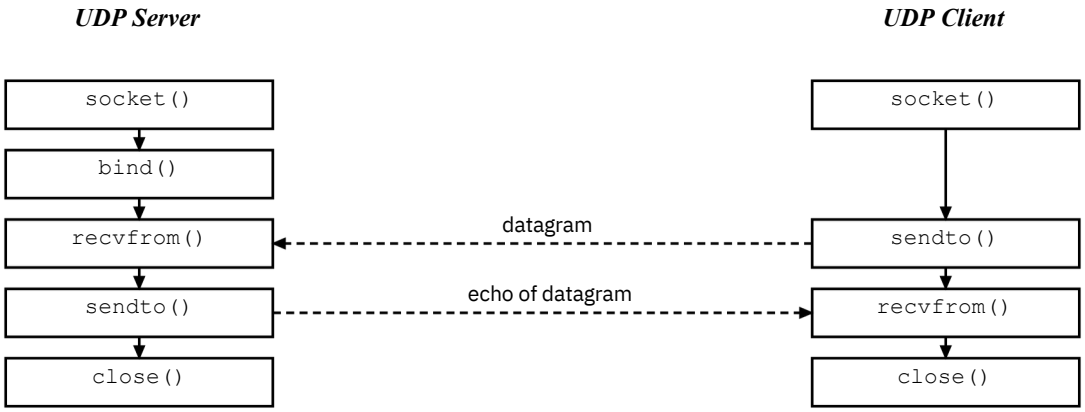


Fig 2: Socket calls in UDP client-server communication

PROGRAM:

echo_tcp_server.py

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 5000))
s.listen(1)
print("TCP Echo Server listening on port 5000 ...")
conn, addr = s.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Received :", data.decode())
    conn.sendall(data)                # echo back
conn.close()
s.close()
print("Connection closed.")
```

echo_tcp_client.py

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 5000))
for msg in ["Hello Server", "Computer Networks Lab", "bye"]:
```

echo_tcp_client.py (contd.)

```
    c.sendall(msg.encode())
    print("Sent     :", msg)
    print("Echoed   :", c.recv(1024).decode())
c.close()
```

echo_udp_server.py

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.1", 6000))
print("UDP Echo Server waiting on port 6000 ...")
for _ in range(2):
    data, addr = s.recvfrom(1024)
    print("Received from", addr, ":", data.decode())
    s.sendto(data, addr)
s.close()
```

echo_udp_client.py

```
import socket
c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
for msg in ["Hello UDP", "Datagram test"]:
    c.sendto(msg.encode(), ("127.0.0.1", 6000))
    data, _ = c.recvfrom(1024)
    print("Sent     :", msg)
    print("Echoed   :", data.decode())
c.close()
```

chat_server.py

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 7000))
s.listen(1)
print("Chat server started. Waiting for client ...")
conn, addr = s.accept()
print("Client connected:", addr)
while True:
    msg = conn.recv(1024).decode()
    if not msg or msg.lower() == "exit":
        break
    print("Client :", msg)
    reply = input("Server : ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break
conn.close()
s.close()
print("Chat ended.")
```

chat_client.py

```
import socket
c = socket.socket()
c.connect(("127.0.0.1", 7000))
print("Connected to chat server. Type 'exit' to quit.")
while True:
    msg = input("Client : ")
    c.send(msg.encode())
    if msg.lower() == "exit":
        break
    reply = c.recv(1024).decode()
    if reply.lower() == "exit":
        break
    print("Server :", reply)
c.close()
print("Chat ended.")
```

OUTPUT:

(a1) TCP echo

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 1 - Server)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 echo_tcp_server.py
TCP Echo Server listening on port 5000 ...
Connected by ('127.0.0.1', 46248)
Received : Hello Server
Received : Computer Networks Lab
Received : bye
Connection closed.
```

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 2 - Client)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 echo_tcp_client.py
Sent     : Hello Server
Echoed   : Hello Server
Sent     : Computer Networks Lab
Echoed   : Computer Networks Lab
Sent     : bye
Echoed   : bye
```

(a2) UDP echo

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 1 - Server)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 echo_udp_server.py
UDP Echo Server waiting on port 6000 ...
Received from ('127.0.0.1', 53205) : Hello UDP
Received from ('127.0.0.1', 53205) : Datagram test
```

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 2 - Client)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 echo_udp_client.py
Sent     : Hello     UDP
Echoed   : Hello     UDP
Sent     : Datagram test
Echoed   : Datagram test
```

(b) Chat program

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 1 - Chat Server)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 chat_server.py
Chat server started. Waiting for client ...
Client connected: ('127.0.0.1', 43260)
Client : Hi Server, this is UusSeErR
Server : Hello UusSeErR , how can I help?
Client : Is the CN lab record ready?
Server : Yes, the lab record is due on Friday.
Client : Thanks, bye
Server : exit
Chat ended.
```

UusSeErR@UusSeErR-VirtualBox: ~ (Terminal 2 - Chat Client)

```
UusSeErR@UusSeErR-VirtualBox:~$ python3 chat_client.py
Connected to chat server. Type 'exit' to quit.
Client : Hi Server, this is UusSeErR
Server : Hello UusSeErR , how can I help?
Client : Is the CN lab record ready?
Server : Yes, the lab record is due on Friday.
Client : Thanks, bye
Chat ended.
```

RESULT:

Thus the echo client-server programs using TCP and UDP sockets and the chat program using socket programming were implemented and executed successfully.